

Scaling Real-Time Traffic Analytics on Edge–Cloud Fabrics for City-Scale Camera Networks *

Akash Sharma¹, Pranjal Naman¹, Roopkatha Banerjee¹, Priyanshu Pansari¹,
Sankalp Gawali¹, Mayank Arya¹, Sharath Chandra¹, Arun Josephraj³,
Rakshit Ramesh³, Punit Rathore^{2,4}, Anirban Chakraborty¹,
Raghu Krishnapuram^{3,4}, Vijay Kovvali⁴ and Yogesh Simmhan^{1,2}

¹Department of Computational and Data Sciences

²Robert Bosch Center for Cyber Physical Systems (RBCCPS)

³Center of Data for Public Good (CDPG)

⁴Centre for infrastructure, Sustainable Transportation & Urban Planning
Indian Institute of Science, Bengaluru, India

Email: {akashsharma, pranjalnaman, simmhan}@iisc.ac.in

Abstract

Real-time city-scale traffic analytics requires processing 100s–1000s of CCTV streams under strict latency, bandwidth, and compute limits. We present a scalable AI-driven Intelligent Transportation System (AIITS) designed to address multi-dimensional scaling on an edge–cloud fabric. Our platform transforms live multi-camera video feeds into a dynamic traffic graph through a DNN inferencing pipeline, complemented by real-time nowcasting and short-horizon forecasting using Spatio-Temporal GNNs. Using a testbed to validate in a Bengaluru neighborhood, we ingest 100+ RTSP feeds from Raspberry Pis, while Jetson Orin edge accelerators perform high-throughput detection and tracking, producing lightweight flow summaries for cloud-based GNN inference. A capacity-aware scheduler orchestrates load-balancing across heterogeneous devices to sustain real-time performance as stream counts increase. To ensure continuous adaptation, we integrate SAM3 foundation-model assisted labeling and Continuous Federated Learning to update DNN detectors on the edge. Experiments show stable ingestion up to 2000 FPS on Jetson Orins, low-latency aggregation, and accurate and scalable ST-GNN forecasts for up to 1000 streams. A planned live demonstration will scale the full pipeline to 1000 streams, showcasing practical, cross-fabric scalability.

*Accepted at TCSC SCALE Challenge 2026. To appear in the Proceedings of IEEE/ACM CCGRID Workshops, Sydney, 2026.

1 Introduction

Social Impact of Traffic Congestion. Traffic congestion and safety remain critical challenges in rapidly urbanizing regions, impacting quality of life, public health, and energy sustainability. This is especially severe in emerging megacities with >10M populations. Bengaluru, India, the second most congested city globally, experiences average commute speeds of 14 kmph (8.7 mph), leading to nearly 7 lost days annually per commuter [21]. Road transport contributes $\approx 30\%$ of global energy use, making congestion a sustainability concern [7]. Air quality also degrades with traffic density [11]. India reports the world’s highest road fatalities, underscoring the need for systemic interventions [6].

An AI-Driven Approach to ITS. These realities motivate the need for *Intelligent Transportation Systems (ITS)* that can operate effectively under heterogeneous, unstructured, and dynamic traffic conditions. Unlike lane-disciplined traffic in developed regions, Indian roads exhibit mixed vehicle types and models, ranging from two-wheelers and three-wheelers to buses and trucks, moving in non-lane-based patterns. Compounding this, developing cities are also under-invested in specialized sensing infrastructure for traffic monitoring, such as lane or ANPR (Automatic Number Plate Recognition) cameras, relying instead on thousands of general-purpose safety cameras that were not designed for traffic flow analytics. As a result, traditional computational traffic simulators (e.g., SUMO, Vissim) cannot be calibrated effectively for these conditions to optimize traffic [17]. These constraints motivate *AI-driven ITS (AIITS)* that infer traffic states from large, heterogeneous camera networks.

Need for Speed in AIITS. AIITS pipelines are computationally and communication-intensive. Bengaluru, our case-study city, has > 5000 safety cameras that generate continuous HD video feeds. Today’s centralized architectures rely on high-bandwidth fiber networks, GPU servers, and petabytes of archival storage for even *manual monitoring*. Transitioning to AI-driven ITS solutions requires *active real-time computation*, not just *passive storage* over the streams. Scaling to 100s of gigabits and 100k+ Frames per Second (FPS) of video streams implies unsustainable growth in cloud GPU servers. Further, actionable traffic forecasting requires low-latency ingestion and inference, which centralized systems struggle to provide across Wide Area Networks (WANs) as camera deployments increase. This defines our problem: *enabling real-time analytics from 1000s of city cameras under strict latency and resource constraints, requiring coordinated scale-out, scale-up and elastic stream distribution*.

Contributions. We introduced a scalable *edge-cloud framework* for real-time AI-driven traffic analytics over city-wide camera networks. Our system processes 100s of heterogeneous video streams under strict latency, bandwidth, and compute constraints by designing an *edge-first design* that performs high-throughput detection and tracking on distributed Jetson accelerators, while Raspberry Pis serve large-scale RTSP video streams. The framework demonstrates *multi-dimensional scalability*: (i) *scale-out* across 50+ heterogeneous edge devices and 100 streams, (ii) *scale-up* of GNN pipelines to sustain real-time inference of up to 1000 streams, and (iii) *cross-fabric scaling* from edge

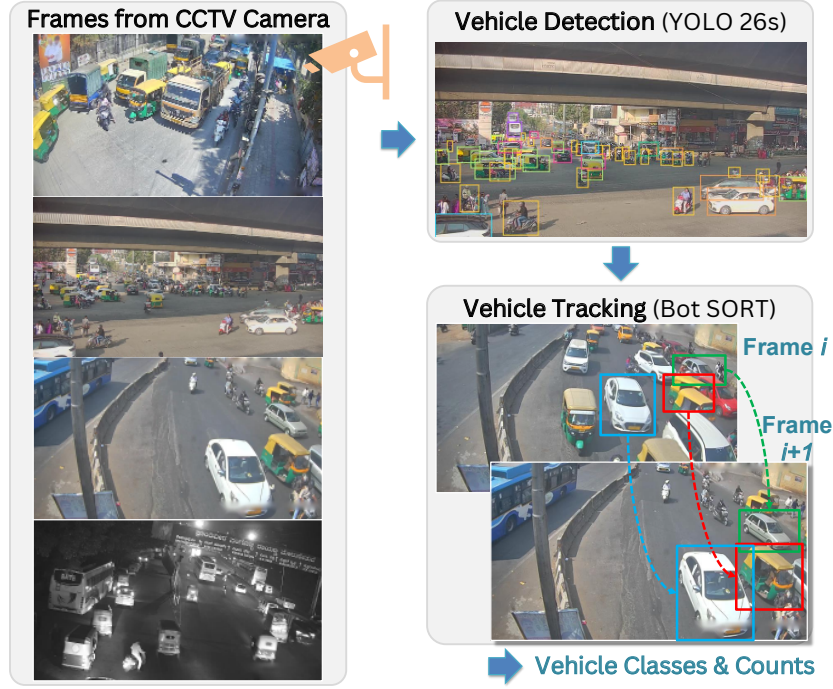


Figure 1: Sample images reflecting complex traffic in Bengaluru, with a DNN pipeline to detect vehicle classes and counts.

to cloud.

A *capacity-aware stream scheduler* intelligently assigns video streams to edge accelerators based on empirical FPS thresholds, to sustain real-time performance as streams scale from tens to hundreds. Structured per-camera traffic flow summaries are fused into a dynamic city-level temporal traffic graph feeding a *GNN-based forecasting* service for short-horizon congestion predictions. To maintain analytical freshness, a *Continuous Federated Learning* pipeline detects *de novo* vehicle classes using a *foundation model* and retrain DNN detectors on the edge, concurrently with inference.

Our end-to-end evaluation for a *neighborhood in Bengaluru* with 100 cameras shows sustained real-time throughput, low latency, and robust scaling. This demonstrates a deployable and socially impactful AIITS platform. We plan to expand this to 1000 streams in a live demonstration.

2 Towards AI-driven ITS in Bengaluru

The *AI for Mobility at IISc (AIM@IISc)* is an inter-disciplinary team that develops scalable, data-driven tools to improve congestion and safety on Indian roads. Bengaluru serves as an ideal real-world testbed, with close engagement from the city traffic police to design and validate novel and scalable methods. We introduce the high-level AIITS requirements here, followed by the proposed design in § 3.

A critical requirement for scalable AIITS is converting 5000+ safety camera streams into a *temporal graph* that exposes real-time network conditions. This transformation

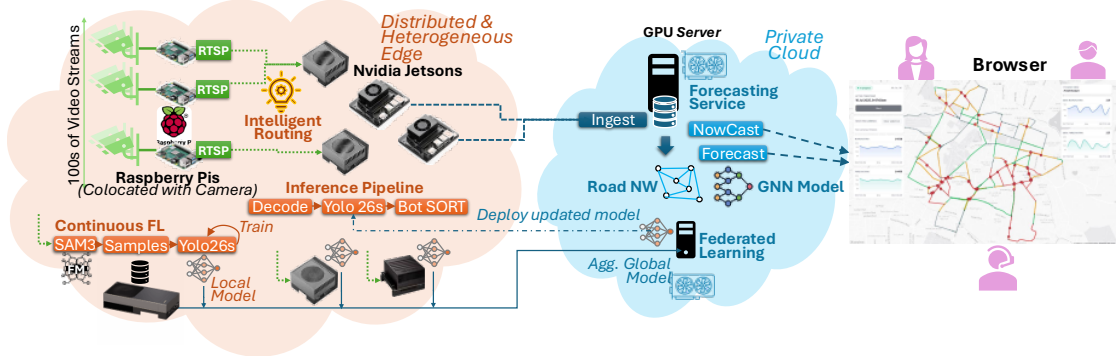


Figure 2: Scalable AIITS Testbed and Architecture for Bengaluru Traffic Camera Feeds.

reduces the high-volume, bandwidth-heavy multi-dimensional data into a concise temporal graph form suitable for downstream analytics. We address this through an edge-first inferencing pipeline (Fig. 1) coupled with cloud-based aggregation.

Multiple cameras predominantly placed at road junctions cover only a fragment of the 2000+ kms road network across a 700 km² region. This requires *spatial extrapolation* from the ≈ 1000 observed vertices and edges to the 10k+ unobserved roads across the city. Given the limitations of calibrating classical traffic simulators for India’s heterogeneous traffic, we adopt *Spatio-temporal Graph Neural Network (ST-GNN)*, enabling both real-time *nowcasting* and *short-term forecasting* of traffic across the broader network.

These forecasts support higher-level analytics that help inform interventions.

Anomaly detection over the predicted temporal graph can flag segments with unusually high congestion, supporting targeted traffic police deployment or remote signal control. Such wide-area, real-time signal optimization can be increasingly automated. The ST-GNN models also support *what-if analysis*, allowing planners to evaluate policy options such as introducing one-way flows, adjusting lane ratios or adding bus-only lanes. These reduce the cost of the solution and promote evidence-driven urban mobility decisions.

Finally, training accurate vision models for Indian traffic is challenging due to heterogeneous vehicle types and evolving conditions. Our prior large-scale annotation effort (e.g., UVH-26 [20]) improved the accuracy of state-of-the-art YOLO and DETR models from 8.4 to 31.5% over equivalent baseline models trained on the COCO dataset [16], but remains human-intensive and cannot scale across cities. This motivates using *multi-modal foundational models*, e.g., SAM3, CLIP, as automatic teachers to bootstrap localized training data. We integrate this into a *Continuous Federated Learning* setup that enables scalable, distributed retraining on edge accelerators, without interrupting real-time inference. This demonstrates scalable application–infrastructure co-design.

3 Scalable AIITS Architecture and Testbed

Fig. 2 illustrates the architecture of our AI-driven ITS platform, which spans edge and cloud resources. We instantiate an *AIITS testbed* that emulates the proposed city-scale deployment at an $\mathcal{O}(100)$ scale stream. Each video camera has an embedded computer that hosts an RTSP server to serve the streams, proxied in our testbed using Raspberry

Pis. The core computational fabric consists of *Nvidia Jetson Edge accelerators*, including Orin AGX and Thor developer kits, which provide multi-core Arm CPUs, 1792–2560 Ampere/Blackwell CUDA cores and 32–128GB of share CPU–GPU LPDDR memory in compact (11cm×11cm×7.2cm, 800–1800 gms) low-powered (60–130W peak) form factor for field deployment. These devices are colocated with cameras at certain roadside units and ingest video streams over the local fiber network to perform real-time inferencing and training.

A small number of centralized GPU workstations, hosted at the City’s private cloud, orchestrate the platform, ingest light-weight structured flow updates, maintain the temporal traffic graph, execute nowcasting and forecasting using GNN models, and manage federated learning aggregation. These also host browser-based services that expose live forecasts, alerts, and system controls to city operators, residents, and other stakeholders. We discuss these various components next.

3.1 Streaming RTSP Streams from Edge

To evaluate our system at scale without relying on physical CCTV deployments, we construct a distributed testbed that emulates a large urban camera network using hundreds of concurrent *Real-Time Streaming Protocol (RTSP)* video streams. Each stream is hosted on a heterogeneous cluster of Raspberry Pis, where every Pi runs *MediaMTX*, an open-source RTSP server. Each server exposes one or more streams through unique endpoints for downstream processing. Video is served using *FFmpeg*, which replays pre-recorded camera footage stored locally on each Pi for specific time durations. By using *FFmpeg*’s *stream-copy*, we avoid transcoding overheads, enabling multiple real-time streams even on low-power Pis. This architecture allows high concurrency across the cluster. Collectively, our 42 Pis of 3B and 4B types are able to serve 100 distributed streams, forming a scalable, low-cost emulation of a city-wide camera network neighborhood. This also weakly-scales with the addition of more Pis to support a larger number of streams.

3.2 Scaling Detection Models on Edge Accelerators

To convert raw multi-camera video streams into data suitable for GNN-based forecasting, we deploy a distributed edge inference layer across heterogeneous Nvidia Jetson devices in our testbed. This layer ingests live RTSP streams, performs real-time detection and tracking, and produces compact vehicle-flow summaries for downstream analytics. By executing inference locally, the system avoids high-bandwidth video transmission to the cloud and reduces end-to-end latency.

3.2.1 Stream Ingestion

End-users select camera streams via a frontend interface, which are then dynamically assigned to Jetson accelerators (§ 3.2.3) by our scheduler. On each Jetson (Fig. 2), hardware-accelerated *GStreamer* pipelines utilize the on-board *nvdec* accelerator to

decode streams directly into GPU-native memory. DeepStream’s *nvstreammux* batches frames from multiple camera streams into unified inference batches, enabling high GPU utilization and allowing Jetsons to process many simultaneous streams at real-time frame rates.

3.2.2 Detection and Tracking

Each batched frame is processed using the YOLO26s model [10], trained on our UVH-26 dataset [20] and deployed using DeepStream’s inference engine. While object detection provides per-frame bounding boxes and classes, real-time traffic estimation requires tracking *unique vehicles* in a feed. To accomplish this, each stream integrates BoT-SORT multi-object tracking [1], which assigns persistent IDs to detected vehicles across successive frames. For every processed frame, the system emits vehicle-level records consisting of a tracking ID, class label, and bounding box. These identity-aware observations are aggregated over short intervals by our pipeline (tunable, 5–30s) and forwarded as low-overhead summaries to the GNN inference pipeline, avoiding the need to transmit raw video or dense feature maps.

3.2.3 Intelligent Routing

Because the inference layer operates over heterogeneous Jetson devices, effective stream placement is essential for maintaining real-time performance. We enable elastic scaling by assigning streams to accelerators using a *capacity-driven scheduling policy*, based on empirically derived per-device limits. For each Jetson model, we profile it offline by progressively increasing the input workload until the device no longer sustains the native frame rate, defining its stable throughput. In our deployment, Orin AGX 32GB and 64GB models support ≈ 200 FPS and 400 FPS, respectively. Stream assignment is then formulated as a bin-packing problem [5], where each stream’s FPS contributes to the item weight and each device represents a bin with known capacity. We evaluate two classic heuristics:

Best Fit Streams are placed on the smallest-capacity device that can still accommodate them. In our testbed devices (§ 4.1), this packs the loads tightly onto the Orin 32GB devices first, and activates the larger Orin 64GB devices only when cumulative demand exceeds ≈ 1000 FPS. This minimizes the number of active devices and reduces overall power consumption at moderate workloads.

Worst Fit Streams are assigned to the device with the largest remaining capacity, distributing the load across even the higher-capacity devices earlier. This improves load balance and thermal distribution (e.g., when it is a hot day and the IP67 packaging imposes thermal limits) but activates more devices sooner, increasing baseline cluster power usage.

These strategies allow the system to flexibly scale detection and tracking across heterogeneous edge accelerators while accounting for power usage and sustaining real-time throughput for large, multi-camera deployments.

3.3 Traffic Forecasting on Temporal Graphs using GNNs

The GPU server hosts the *ingest*, *nowcast* and *forecast services* that operate on the structured vehicle-flow outputs received from the edge inference layer (Fig. 2). This represents the scale-up aspect of our architecture, leveraging GPU servers for high-throughput graph inference. A local GPU workstation in our testbed mimics an accelerated VM in the private cloud. The *ingest service* receives per-camera vehicle counts at 1-second granularity, batched every 15 seconds, and maintains an on-disk time-series database with timestamp, camera ID, and a class-count vector. The *nowcast service* exposes the aggregated traffic state to downstream applications through a low-latency streaming interface implemented as a gRPC service.

The *forecast service* applies a *Spatio-Temporal Graph Neural Network (ST-GNN)* to predict short-horizon traffic conditions across the road network. ST-GNNs jointly model the spatial structure of road graphs and the temporal evolution of traffic patterns. We use the TrendGCN architecture [9], which integrates vertex embeddings, temporal embeddings and a graph-convolutional GRU to capture dynamic traffic correlations across junctions. It is trained offline by aggregating tracker outputs into 1-minute junction-level vehicle-count time series and using 180 hours of historical data from 100 junctions with a lag of 5 minutes and a 5-minute forecast horizon. The model queries historical data from the ingest database based on a configurable lag window (e.g., several minutes), and outputs predicted vehicle counts for each junction over the chosen forecast horizon.

Because camera coverage is incomplete, only ≈ 100 of the over 250 junctions are observed in our validation neighborhood; we forecast on a coarsened graph that retains only camera-equipped junctions as nodes. Edges in this reduced graph are super-edges formed by collapsing sequences of road segments between observed junctions, preserving large-scale structure for short-range traffic propagation while ensuring reliable inputs at every node [15]. To obtain street-level (edge) flow estimates from predicted junction-level counts, we apply a mass-conserving allocation that distributes vehicle volumes across adjacent super-edges proportional to their connectivity, and aggregate endpoint contributions to compute edge counts, maintaining consistency between vertex and edge predictions for congestion estimation.

Finally, edge-level flows are discretized into congestion states (e.g., free-flow, moderate, heavy) for dashboard integration and downstream analytics. By combining high-frequency edge summaries, spatio-temporal forecasting, and graph-based abstractions, the system delivers accurate short-horizon predictions that scale to city-wide deployments under partial camera coverage.

3.4 LLM-driven Labeling for Federated Learning on Edge

Edge-based detection models must continuously adapt to highly localized and evolving vehicle categories, such as auto-rickshaws (3-wheelers) or emerging commercial vehicle types, that are not well represented in standard datasets. To support this, we implement a continuous FL workflow across K Jetson devices, enabling each edge device to curate, annotate, and train on its own data without transmitting raw video to a central server. Each Jetson collects training samples through temporally stratified sampling. Frames are accumulated over a configurable window to form a local, diverse dataset.

After the collection phase, each device uses the SAM3 foundation model [3] to automatically annotate the sampled images. The annotation pipeline accepts an image and a predefined set of text prompts, e.g.,

$C = \{“a sedan”, “a sport-utility vehicle”, \dots\}$, which cover common and region-specific vehicle classes, and a subset of which may be unknown to the object detection model. These prompts are tokenized into embeddings E_c and the SAM3 model f_θ processes these inputs to generate bounding box coordinates B_c and raw class logits Z_c for a batch of data $P: (Z_c, B_c) = f_\theta(P; E_c)$. Logits are converted to probabilities using a sigmoid function, and only predictions exceeding a confidence threshold τ (e.g., $\tau = 0.30$) are retained. The resulting per-device dataset is $D_k = \bigcup_{c \in C} \{(c, \text{bbox}_q, p_q) \mid p_q(c) \geq \tau\}$.

Each Jetson then performs local fine-tuning of the YOLO-based detector, initialized with the current global weights. After E local epochs of training over D_k , devices return their updated weights to the server, which aggregates them using Federated Averaging (FedAvg). This aggregated global model is then broadcast back to the edge devices for use in the detection inference pipeline.

3.5 Dashboard

We implement an interactive React+Mapbox dashboard (Fig. 2, right) as the system’s primary visualization and control interface. Outputs are overlaid on an OSM map of Bengaluru, with camera locations and inferred road segments rendered geospatially. Updates stream via gRPC to a FastAPI backend and are forwarded to the browser over WebSockets to maintain low-latency, continuous refresh. The map provides a live nowcast view, showing how per-camera vehicle-flow summaries translate into city-level traffic conditions. Color-coded overlays indicate congestion levels, enabling operators to assess the network state at a glance. A complementary time-series panel visualizes real-time and forecasted vehicle counts, updated as the ingestion and forecasting services produce new outputs. Displaying short-horizon GNN forecasts alongside current measurements helps highlight emerging congestion trends.

The dashboard also supports lightweight orchestration. Users can select individual cameras or subsets of streams to trigger targeted execution of the detection and tracking pipeline. Selecting a road segment opens a historical summary of vehicle flow, derived from second-wise aggregated counts maintained by the ingest service.

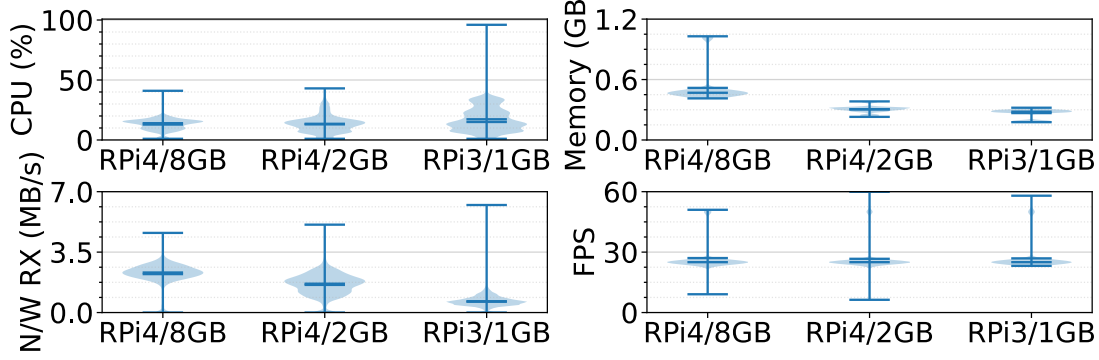


Figure 3: Performance of RPi hosting 100 RTSP Streams

4 Results

4.1 Setup

Our RPi cluster hosts 1000 camera streams, with 10 Pi 4B/8GB with 4 streams each, 17 Pi 4B/2GB with 2–33 streams each, , and 15 Pi 3B/1GB with 1 stream. Video feeds are usually at 25 FPS, pre-synchronized across Pis and streamed by the RTSP servers. The testbed also has an accelerated edge cluster Jetson Orin AGX, five with 32GB and 4 with 64GB, to host the inference pipelines, and ingest these 100 video streams. The FL setup runs on these same devices

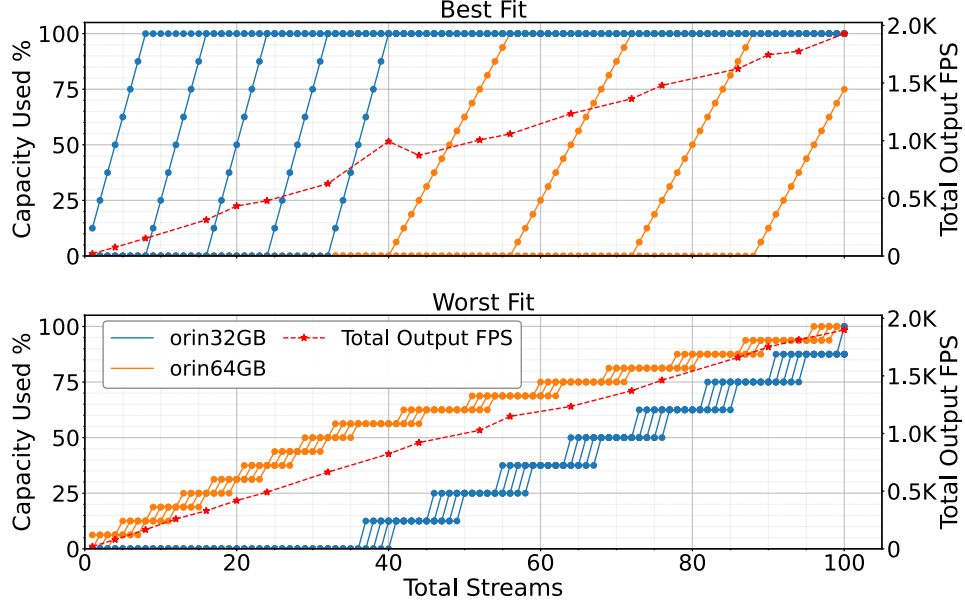
4.2 Scalability of Edge Analytics for Vehicle Counting

4.2.1 RPi RTSP Stream Performance

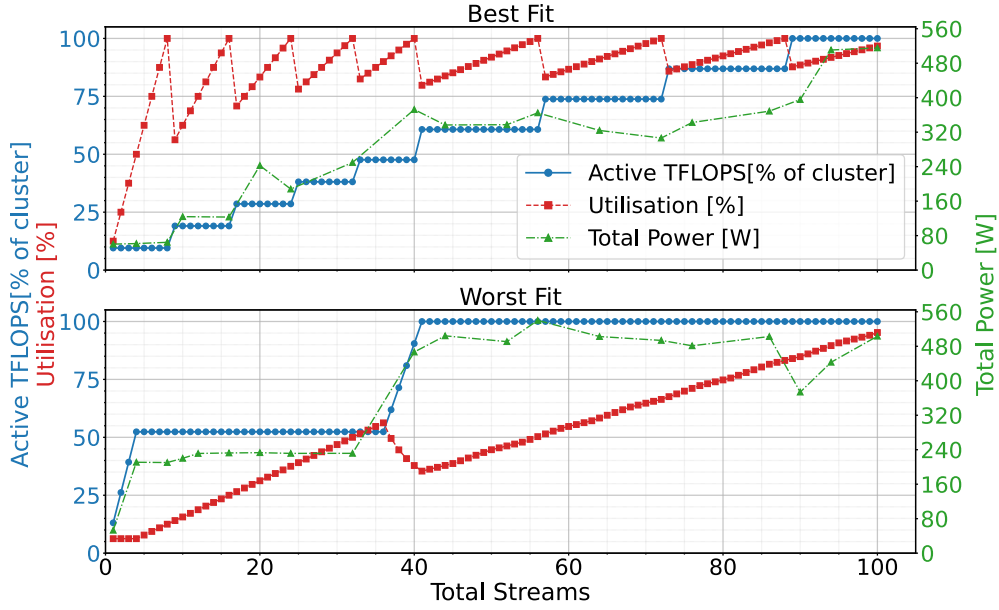
To validate testbed stability, we evaluate resource usage across Raspberry Pi variants (Fig. 3). The results show that overhead on these RTSP publisher devices is minimal: median CPU utilization stays below 25% on all models, largely due to the stream-copy mechanism avoiding costly encode/decode operations. Memory usage remains modest as well, peaking at 30% on the RPi 3/1GB, with the RPi4/8GB slightly higher because it hosts four streams. Network throughput follows stream count: RPi 4/8GB sends the most data, while RPi 4/2GB send two or three streams, but all remain within bandwidth limits at ≤ 7 MB/s, below the 100Mbps or 12.5 MB/s cap of RPi3. FPS remains stable, within 25 ± 1 FPS, $> 90\%$ of the time. These results confirm that even low-power Pis can reliably scale to high stream densities for our AIITS testbed without degrading stream quality.

4.2.2 Edge Analytics

We evaluate the scalability of edge analytics on the testbed cluster of Nvidia Jetson devices by increasing the number of ingested RTSP streams. As expected, *cumulative output FPS* processed scales with input load as long as no device exceeds its empirically determined FPS capacity (§ 3.2.3), ensuring real-time performance even with over 2000 frames processed per second (Fig. 4a). To further quantify performance and power



(a) FPS Capacity Use% and Cumulative FPS Processed



(b) Active and Used TFLOPS% in cluster, and Total Power Draw

Figure 4: Efficiency and Scaling of Jetsons with streams ingested

efficiency, we track the *Total Active Capacity* (sum of TOPS of active devices) and *Utilization* (load relative to per-device FPS capacity), shown in Fig. 4b, alongside *total power draw*. We compare the two stream-assignment strategies.

Best Fit (upper plots) tightly packs streams onto the smallest suitable device, saturating the 32 GB Orins before activating the 64 GB ones. This produces a stepwise activation pattern, minimizes the number of active devices at moderate workloads, and reduces power consumption by deferring high-capacity nodes. *Worst Fit* (lower plots)

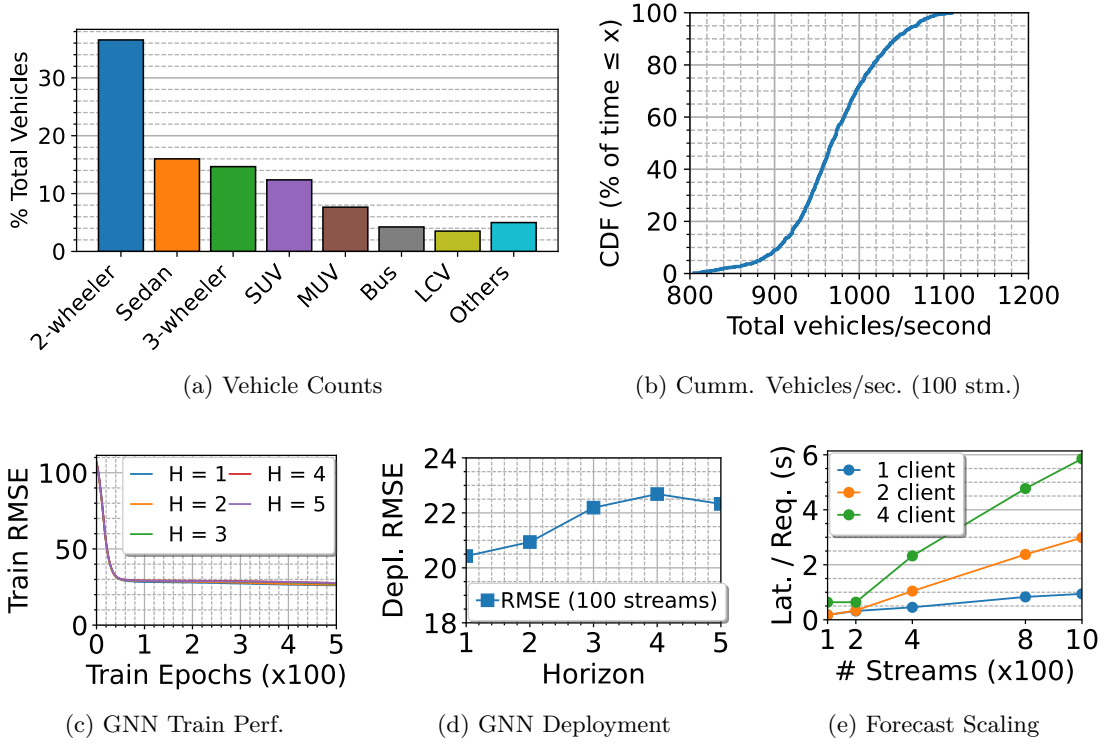


Figure 5: Ingest Rate and GNN Performance

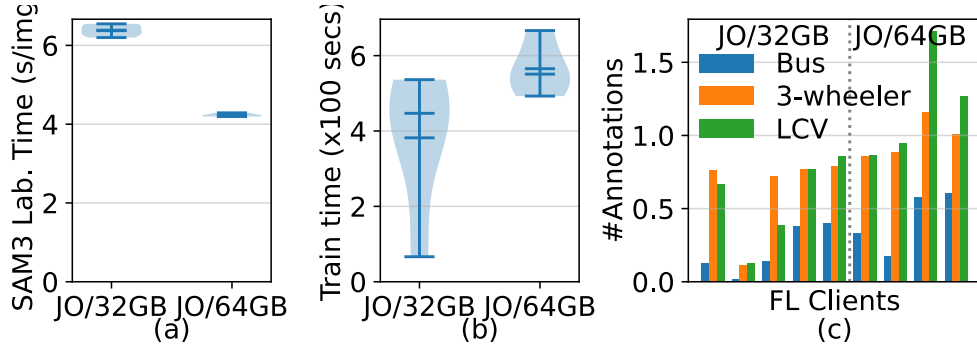


Figure 6: Performance of SAM3 labeling and FL training.

distributes load across devices with the most remaining capacity, engaging larger devices earlier and improving load balance. However, this activates more devices sooner, increasing baseline power use. That said, in a heterogeneous cluster, the effect is more nuanced: Worst Fit may preferentially load high-capacity devices with better power-per-stream efficiency, *sometimes yielding comparable or even lower power than Best Fit*. E.g., at 32 streams, Worst Fit consumes 231.6W versus 249.6W for Best Fit. These results show that both policies maintain real-time throughput, with Best Fit optimizing for minimal activation and Worst Fit improving balance. and demonstrating effective scale-out across heterogeneous edge accelerators.

4.3 Scalability of Ingest and ST-GNN Forecasting

We analyze vehicle-flow data from the ingest service alongside the ST-GNN forecasting pipeline. Across 100 streams over 15 minutes, the system captures a diverse vehicle mix, with two-wheelers (37%), sedans (15%), and three-wheelers (14%) forming the majority (Fig. 5a). Aggregate detections exceed 1000 unique vehicles/s for nearly 30% of the window, peaking at 1110 vehicles/s, directly reflecting ingest throughput and exhibiting scalability under sustained load (Fig. 5b). TrendGCN training converges quickly across all horizons, with RMSE stabilizing early (Fig. 5c). In deployment, RMSE increases modestly from 20 at a 1-minute horizon to approximately 23 at 4 minutes and beyond, indicating stable multi-step forecasting performance (Fig. 5d). To assess scalability, we increase stream count from 100 to 1000 (augmenting with synthetic streams). With forecasts generated every 5 seconds, latencies remain manageable as concurrent clients scale from 1 to 4 (Fig. 5e), confirming that the ST-GNN pipeline sustains both higher camera volumes and client demand.

4.4 Scalability of FM and Continuous FL

To evaluate the practicality and scalability of the FL framework, we deploy the system on a cluster of 9 *Nvidia Jetson Orin AGX*, $5 \times 32\text{GB}$ (JO/32GB) and $4 \times 64\text{GB}$ (JO/64GB). These Jetson devices collectively ingest 300 concurrent video streams, with each JO/32GB hosting 28 streams and JO/64GB hosting 40 streams. As detailed in §3.4, we employ a temporally stratified sampling to extract one frame randomly every 20-second window, over 150 minutes, accruing 45 frames per video stream, totaling 1260 frames total for each JO/32GB and 1800 frames total for each JO/64GB. These frames are labeled by SAM3 leveraging the Orin’s accelerators, with average per-image annotation latencies being 6.3s (JO/32GB) and 4.0s (JO/64GB), as seen in Fig. 6(left). For each of the 9 FL clients, we show the distribution of the three classes unknown to the YOLO model in Fig. 6 (right) and the corresponding train time distribution for each Jetson type is reported in Fig. 6 (center). Since the JO/64GB receives a larger number of data streams, it collects more data, collecting about $1.2\text{--}5\times$ the data collected by the JO/32GB, processing more data per epoch, with a marginally higher training time. The annotation distribution shows non-IIDness of the distribution, highlighting the need for FL in this scenario to ensure all Jetsons are able to recognize all classes.

4.5 Discussion and Scaling Demo

Our AIITS architecture achieves extensibility by functionally isolating edge-inference from cloud-based spatio-temporal forecasting. This facilitates horizontal scale-out of accelerators without systemic reconfiguration. Our approach leverages a novel capacity-aware elastic scheduler and SAM3-driven labeling to mitigate “data drift” expected within unstructured urban environments. By localizing heavy video analytics at the network periphery, the system avoids centralized bandwidth bottlenecks, enabling sustainable, city-scale traffic sensing.

The SCALE Challenge live demo will exhibit the full scalability of our system by expanding beyond the paper’s results to support 1000 video streams across ≈ 100 RPiS and ≈ 25 Jetson accelerators. This will highlight the ability of our design to sustain city-scale, heterogeneous, multi-modal workloads in real time.

5 Related Work

Intelligent Transportation Systems (ITS) often rely on dedicated roadway sensors (e.g., inductive loops) that directly provide usable time-series data but are costly to scale city-wide in emerging nations [24]. While CCTV networks are widespread, converting large volumes of video feeds into traffic-flow signals is less explored and requires distributed multi-stream detection, tracking, synchronization, and aggregation [2, 23]. Prior systems address individual components rather than providing an end-to-end pipeline with an integrated solution, as we do. Li et al. [13] propose a multi-stage surveillance framework but with limited scaling; Seo et al. [19] optimize DNN partitioning but not concurrent multi-stream workloads; while Rong et al. [18] study edge-cloud scheduling under homogeneous assumptions.

Spatio-temporal GNNs [8, 9, 12] achieve strong forecasting performance but assume synchronized sensor inputs rather than video-derived signals produced at scale. A further gap lies in model adaptation. Pretrained detectors often underperform in heterogeneous and emerging city traffic, while constructing large annotated datasets is costly. Open-world discovery methods reduce manual labeling but typically assume centralized data access [22]. FL enables local training on the edge while avoiding raw data sharing [4], yet device heterogeneity remains challenging [14]. Motivated by these gaps, we integrate heterogeneous multi-stream edge inference, capacity-aware scheduling, graph-level aggregation, GNN forecasting and SAM-assisted labeling with lightweight FL updates into a unified end-to-end analytics pipeline for city-scale predictions.

6 Conclusions

We presented an AIITS platform that combines lightweight DNNs, ST-GNN forecasting, foundation-model-assisted labeling, and continuous FL to scale efficiently across an edge-cloud fabric while minimizing data movement and centralized compute. The results confirm its extensibility across large camera deployments and heterogeneous edge hardware, with multi-modal scaling: stream-level scale-out, device-level scale-up, and cross-fabric coordination, demonstrating its practical impact on city-scale urban mobility analytics. In the future, we will evaluate performance under more congested conditions, extend the pipeline to 5000+ feeds, and study the performance interplay between inference and FL. We plan to further optimize stream to Jetson placement using network topology and energy signals, and enable dynamic model selection to sustain throughput with variable streams.

7 Acknowledgments

We thank the Bengaluru Traffic Police (BTP) and the Bengaluru Police for providing access to the Safe City camera data from which the video data used in this study were derived.

References

- [1] Nir Aharon et al. Bot-sort: Robust associations multi-pedestrian tracking. Technical Report 2206.14651, arXiv, 2022.
- [2] Johan Barthélemy, Nicolas Verstaëvel, Hugh Forehead, and Pascal Perez. Edge-computing video analytics for real-time traffic monitoring in a smart city. *Sensors*, 19(9):2048, 2019.
- [3] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.
- [4] H. Brendan McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.
- [5] Edward G. Coffman, Michael R. Garey, and David S. Johnson. Approximation algorithms for bin-packing — an updated survey. In Giorgio Ausiello, Mario Lucertini, and Paolo Serafini, editors, *Algorithm Design for Computer System Design*, volume 284 of *International Centre for Mechanical Sciences*. Springer, Vienna, 1984.
- [6] Global Road Safety Facility (GRSF). GRSF annual report 2024. Technical report, The Global Road Safety Facility, World Bank, 2024. Accessed: Feb. 19, 2026.
- [7] International Energy Agency (IEA). Energy efficiency 2025: Transport. <https://www.iea.org/reports/energy-efficiency-2025/transport>, 2025. Accessed: Feb. 19, 2026.
- [8] Jiawei Jiang, Chengkai Han, Wayne Xin Zhao, and Jingyuan Wang. Pdformer: Propagation delay-aware dynamic long-range transformer for traffic flow prediction. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 4365–4373, 2023.
- [9] Juyong Jiang, Binqing Wu, Ling Chen, Kai Zhang, and Sunghun Kim. Enhancing the robustness via adversarial learning and joint spatial-temporal embeddings in traffic forecasting. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM '23*, page 987–996, New York, NY, USA, 2023. Association for Computing Machinery.

- [10] Glenn Jocher and Jing Qiu. Ultralytics yolo. <https://github.com/ultralytics/ultralytics>, 2023.
- [11] Kanishka Kacker, Ridhima Gupta, and Saif Ali. Does traffic congestion pose health hazards? evidence from a highly congested and polluted city. Discussion Paper Series DP 23-10, Environment for Development (EfD) Initiative, July 2023. Accessed: Feb. 19, 2026.
- [12] Shiyong Lan, Yitong Ma, Weikang Huang, Wenwu Wang, Hongyu Yang, and Pyang Li. Dstagnn: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting. In *International conference on machine learning*, pages 11906–11917. PMLR, 2022.
- [13] Ran Li. A multi stage deep learning approach for real-time vehicle detection, tracking, and speed measurement in intelligent transportation systems. *Scientific reports*, 15(1):22531, 2025.
- [14] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 2020.
- [15] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations*, 2018.
- [16] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.
- [17] Htet Naing, Wentong Cai, Jinqiang Yu, Jinghui Zhong, and Liang Yu. Fine-grained trajectory reconstruction by microscopic traffic simulation with dynamic data-driven evolutionary optimization. *IEEE Transactions on Intelligent Transportation Systems*, 26(2):1930–1950, 2024.
- [18] Chenghao Rong, Jessie Hui Wang, Juncai Liu, Jilong Wang, Fenghua Li, and Xiaolei Huang. Scheduling massive camera streams to optimize large-scale live video analytics. *IEEE/ACM Transactions on Networking*, 30(2):867–880, 2021.
- [19] Woobean Seo, Saehwa Kim, and Seongsoo Hong. Dnnpipe: Dynamic programming-based optimal dnn partitioning for pipelined inference on iot networks. *Journal of Systems Architecture*, 166:103462, 2025.
- [20] Akash Sharma, Chinmay Mhatre, Sankalp Gawali, Ruthvik Bokkasam, Brij Kishore, Vishwajeet Pattanaik, Tarun Rambha, Abdul R. Pinjari, Vijay Kovvali, Anirban Chakraborty, Punit Rathore, Raghu Krishnapuram, and Yogesh Simmhan. The urban vision hackathon dataset and models: Towards image annotations and accurate

vision models for indian traffic. Technical Report arXiv:2511.02563, Indian Institute of Science, 2025.

- [21] TomTom. TomTom traffic index ranking 2025. <https://www.tomtom.com/traffic-index/ranking>, 2025. Accessed: 2026-02-19.
- [22] S. Vaze, V. Balasubramanian, M. Gygli, Y. Nestarev, and A. Vedaldi. Generalized category discovery. In *CVPR*, 2022.
- [23] Hao Frank Yang, Jiarui Cai, Chenxi Liu, Ruimin Ke, and Yinhai Wang. Cooperative multi-camera vehicle tracking and traffic surveillance with edge artificial intelligence and representation learning. *Transportation research part C: emerging technologies*, 148:103982, 2023.
- [24] Xuan Zhou, Ruimin Ke, Hao Yang, and Chenxi Liu. When intelligent transportation systems sensing meets edge computing: Vision and challenges. *Applied Sciences*, 11(20):9680, 2021.